

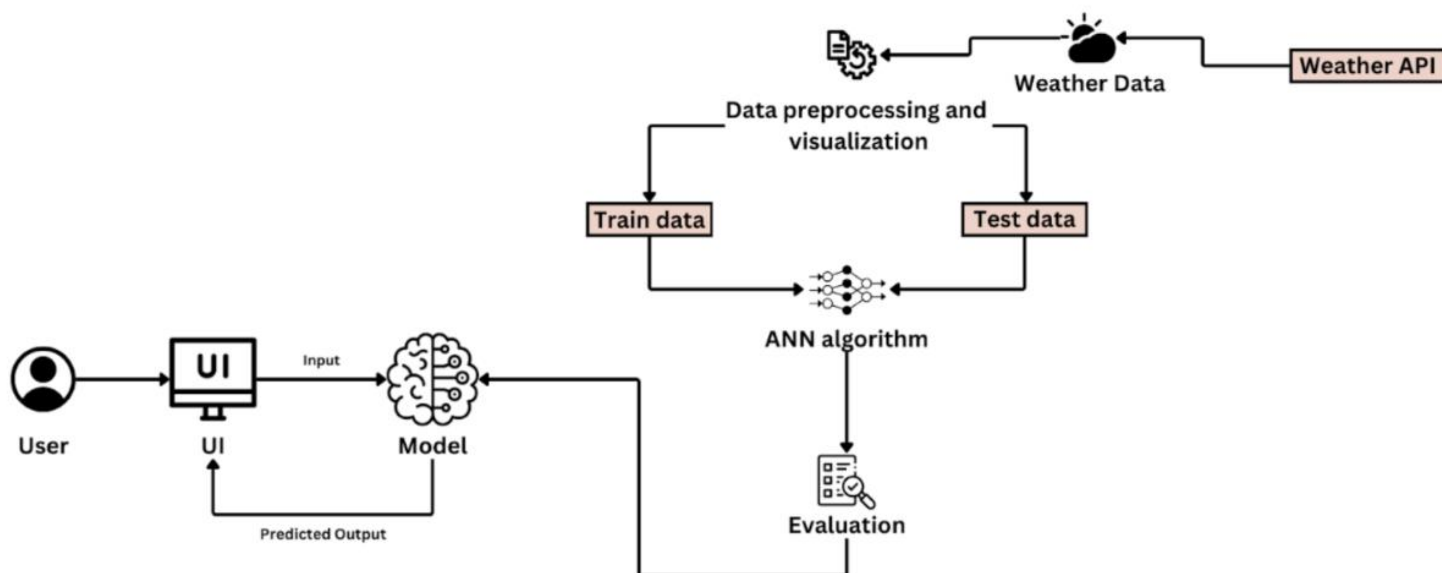
Rainfall Prediction

Team ID: LTVIP2026TMIDS79385

Project Description:

As one of the world's most rainfall-prone countries, India relies heavily on accurate rainfall prediction. The livelihoods of numerous workers and the public's daily lives are closely intertwined with rainfall patterns. Ensuring safety, preparedness, and awareness is a critical need. This project addresses this need by exploring various predictive models, including XGBoost, Support Vector Classification (SVC), Logistic Regression, and Artificial Neural Networks (ANN). The goal is to select the most effective model and make it accessible to the public. The project's impact lies in delivering dependable rainfall predictions that empower individuals across India to make informed decisions, enhancing their safety, economic well-being, and overall quality of life.

Technical Architecture:



Pre-requisites:

1. Visual Studio Code
 2. Python Packages: Flask-> `pip install flask`
XGBoost-> `pip install xgboost`
- Type the given commands in VSCode powershell

Project Objectives:

1. Understand fundamental concepts of Logistic Regression, XGBoost and ANN
2. Know how to preprocess and clean the data
3. Understand how to build a simple web application using Flask

Project Flow:

1. The user interacts with UI which prompts the user to enter necessary values for the prediction
2. The details are sent to the model, which uses it to predict if it will rain or not. The model is integrated into the back-end of the flask file
3. The chosen model utilizes the user's input to predict whether rain is expected or not, and subsequently, displays the prediction on the Flask UI.

For the above workflow to be accomplished, we need to perform the following steps:

- Data Collection:
<https://www.kaggle.com/datasets/trisha2094/weatheraus/>
- Data Cleaning and Preprocessing:
 - o Import libraries-> pandas, numPy, seaborn
 - o Import dataset
 - o Check for null values and handle them
 - o Perform Data visualisation
 - o Handle outliers
 - o Scale the data if needed
 - o Split into train and test
- Model Building:
 - o Import model building libraries
 - o Initialize the models
 - o Fit the model with the data
 - o Evaluate all the models' performances'
 - o Choose the best-performing model
 - o Download the model as a pickle file
- Application building:
 - o Create a front-end file using HTML and CSS
 - o Create a backend file using Python Flask

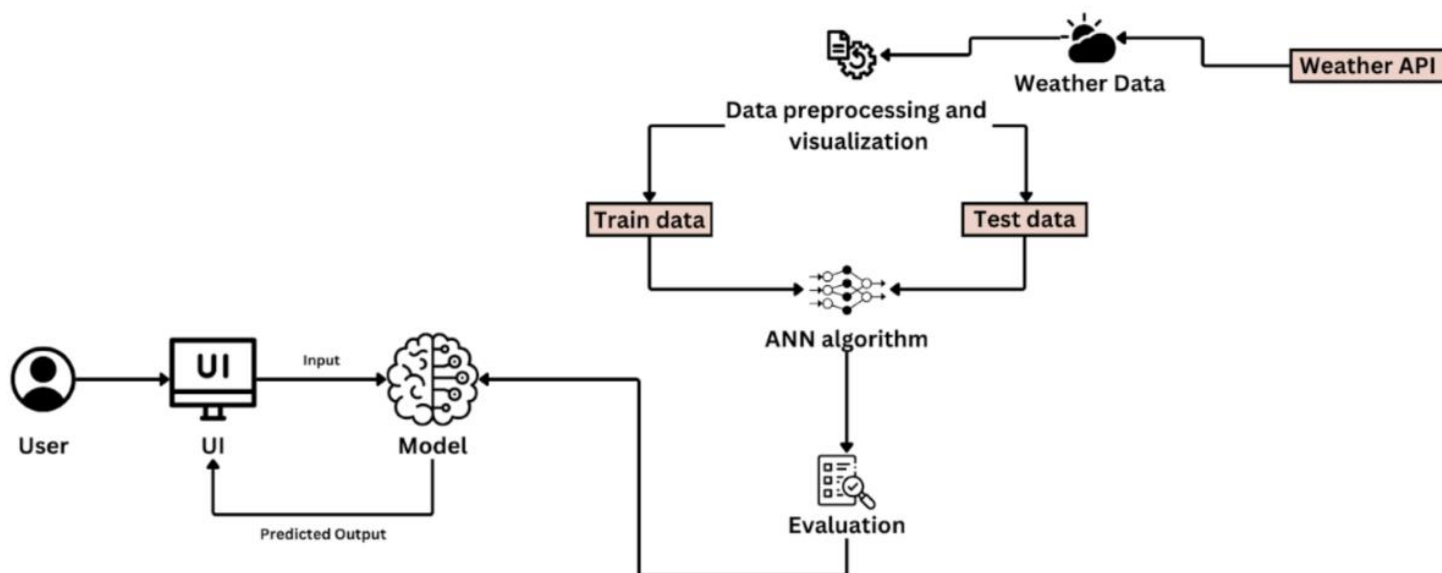
Rainfall Prediction

Team ID: LTVIP2026TMIDS79385

Project Description:

As one of the world's most rainfall-prone countries, India relies heavily on accurate rainfall prediction. The livelihoods of numerous workers and the public's daily lives are closely intertwined with rainfall patterns. Ensuring safety, preparedness, and awareness is a critical need. This project addresses this need by exploring various predictive models, including XGBoost, Support Vector Classification (SVC), Logistic Regression, and Artificial Neural Networks (ANN). The goal is to select the most effective model and make it accessible to the public. The project's impact lies in delivering dependable rainfall predictions that empower individuals across India to make informed decisions, enhancing their safety, economic well-being, and overall quality of life.

Technical Architecture:



Pre-requisites:

1. Visual Studio Code
 2. Python Packages: Flask-> `pip install flask`
XGBoost-> `pip install xgboost`
- Type the given commands in VSCode powershell

3) Analyse the data:

 `df.info()`

 `<class 'pandas.core.frame.DataFrame'>`
RangeIndex: 142193 entries, 0 to 142192
Data columns (total 23 columns):

#	Column	Non-Null Count	Dtype
0	Date	142193 non-null	object
1	Location	142193 non-null	object
2	MinTemp	141556 non-null	float64
3	MaxTemp	141871 non-null	float64
4	Rainfall	140787 non-null	float64
5	Evaporation	81350 non-null	float64
6	Sunshine	74377 non-null	float64
7	WindGustDir	132863 non-null	object
8	WindGustSpeed	132923 non-null	float64
9	WindDir9am	132180 non-null	object
10	WindDir3pm	138415 non-null	object
11	WindSpeed9am	140845 non-null	float64
12	WindSpeed3pm	139563 non-null	float64
13	Humidity9am	140419 non-null	float64

```
[ ] numerical_attributes=[f for f in df.columns if df[f].dtypes!='O']
    categorical_attributes=[f for f in df.columns if df[f].dtypes=='O']
    discrete_val_attributes=[f for f in numerical_attributes if df[f].nunique()<25]
    continuous_val_attributes=[f for f in numerical_attributes if f not in discrete_val_attributes]
    print('numerical_attributes:',numerical_attributes)
    print('categorical_attributes:',categorical_attributes)
    print('discrete_val_attributes:',discrete_val_attributes)
    print('continuous_val_attributes:',continuous_val_attributes)
```

```
numerical_attributes: ['MinTemp', 'MaxTemp', 'Rainfall', 'Evaporation', 'Sunshine', 'WindGustSpeed', 'WindSpeed9am', 'WindSpeed3pm', 'Humidity9am']
categorical_attributes: ['Date', 'Location', 'WindGustDir', 'WindDir9am', 'WindDir3pm', 'RainToday', 'RainTomorrow']
discrete_val_attributes: ['Cloud9am', 'Cloud3pm']
continuous_val_attributes: ['MinTemp', 'MaxTemp', 'Rainfall', 'Evaporation', 'Sunshine', 'WindGustSpeed', 'WindSpeed9am', 'WindSpeed3pm']
```

4) Handle missing values:

```
df.isnull().sum()
```

Date	0
Location	0
MinTemp	637
MaxTemp	322
Rainfall	1406
Evaporation	60843
Sunshine	67816
WindGustDir	9330
WindGustSpeed	9270
WindDir9am	10013
WindDir3pm	3778
WindSpeed9am	1348
WindSpeed3pm	2630
Humidity9am	1774
Humidity3pm	3610
Pressure9am	14014
Pressure3pm	13981
Cloud9am	53657
Cloud3pm	57094

Method to fill in missing values is done by observing the distribution of each attribute and deciding which one of mean, mode and median is most suitable.

```
#The 75th percentile (Q3) and maximum values are relatively close for MinTemp,MaxTemp,Sunshine,Humidity9a
#Pressure9am,Pressure3pm,Cloud9am,Cloud3pm,Temp9am,Temp3pm
#this suggests that the data may not have an extremely skewed distribution.
import matplotlib.pyplot as plt
import seaborn as sns
thresh=22
small_dff=[]
for f in continuous_val_attributes:
    diff=df[f].max()-df[f].quantile(0.75)
    if diff<=thresh:
        small_dff.append(f)
print(small_dff)
```

```
[ ] fig,axes=plt.subplots(3,3,figsize=(15,10))
for i in range(len(small_dff)):
    att=small_dff[i]
    sns.distplot(df[att],ax=axes[i//3,i%3])
```

```
[ ] #MinTemp, MaxTemp,Temp9am and Temp3pm seem to have a normal distribution and thus we'll impute the null values with the mean
#the others will be imputed using median
norm=['MinTemp','MaxTemp','Temp9am','Temp3pm']
for i in norm:
    df[i]=df[i].fillna(df[i].mean())
```

```
df.isnull().sum()
```

```
[ ] rest=[]
for i in continuous_val_attributes:
    if i not in norm:
        rest.append(i)
print(rest)
```

```
[ ] fig,axes=plt.subplots(5,2,figsize=(15,20))
fig.tight_layout(pad=3.0) #padding for better spacing
for i in range(len(rest)):
    att=rest[i]
    sns.boxplot(df[att],ax=axes[i//2,i%2])
    axes[i//2,i%2].set_title(att)
plt.show()
```

```
for i in rest:
    df[i]=df[i].fillna(df[i].median())
```

+ Co

```
[ ] df.isnull().sum()
```

```
[ ] discrete_val_attributes
```

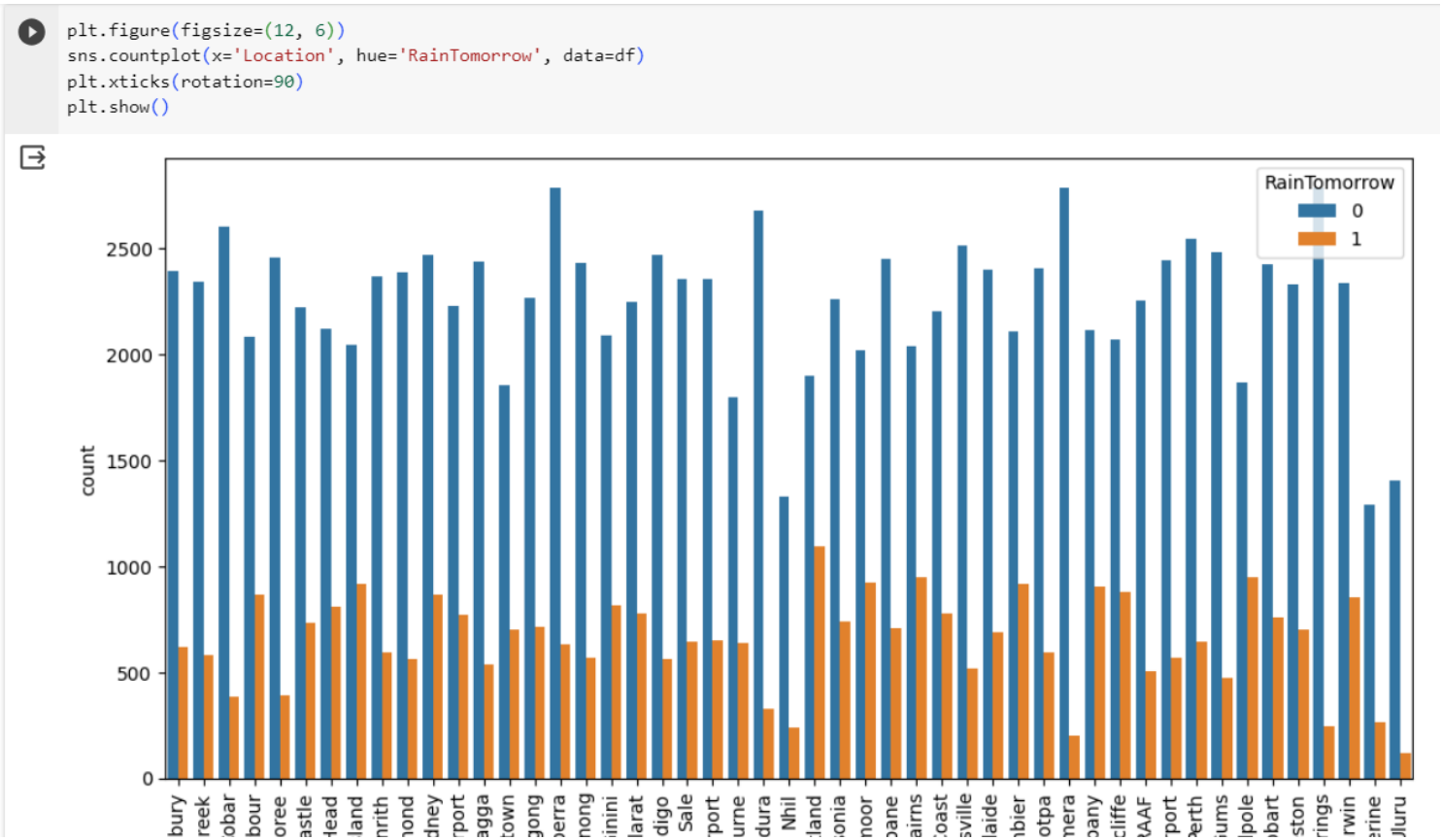
```
[ ] for f in discrete_val_attributes:
    df[f]=df[f].fillna(df[f].mode()[0])
```

```
[ ] df[discrete_val_attributes].isnull().sum()
```

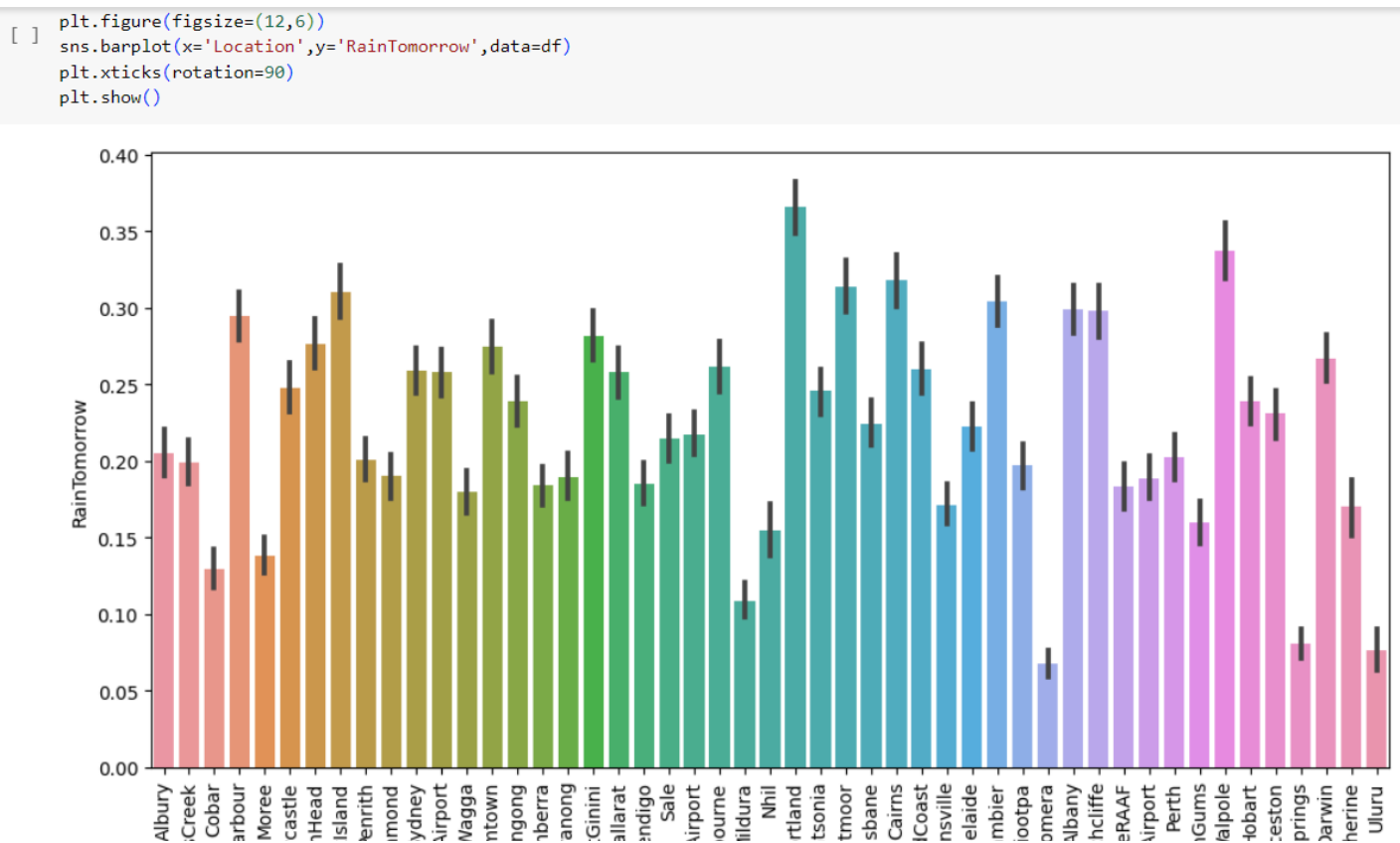
```
[ ] for i in ['WindGustDir','WindDir9am','WindDir3pm','RainToday']:
    df[i].fillna(df[i].mode()[0],inplace=True)
```

- 5) Perform data visualization to understand how change in the attributes affect the target variable. This can be done by find the correlation matrix, printing the heatmap and analyzing the pairplots.

a. Countplot



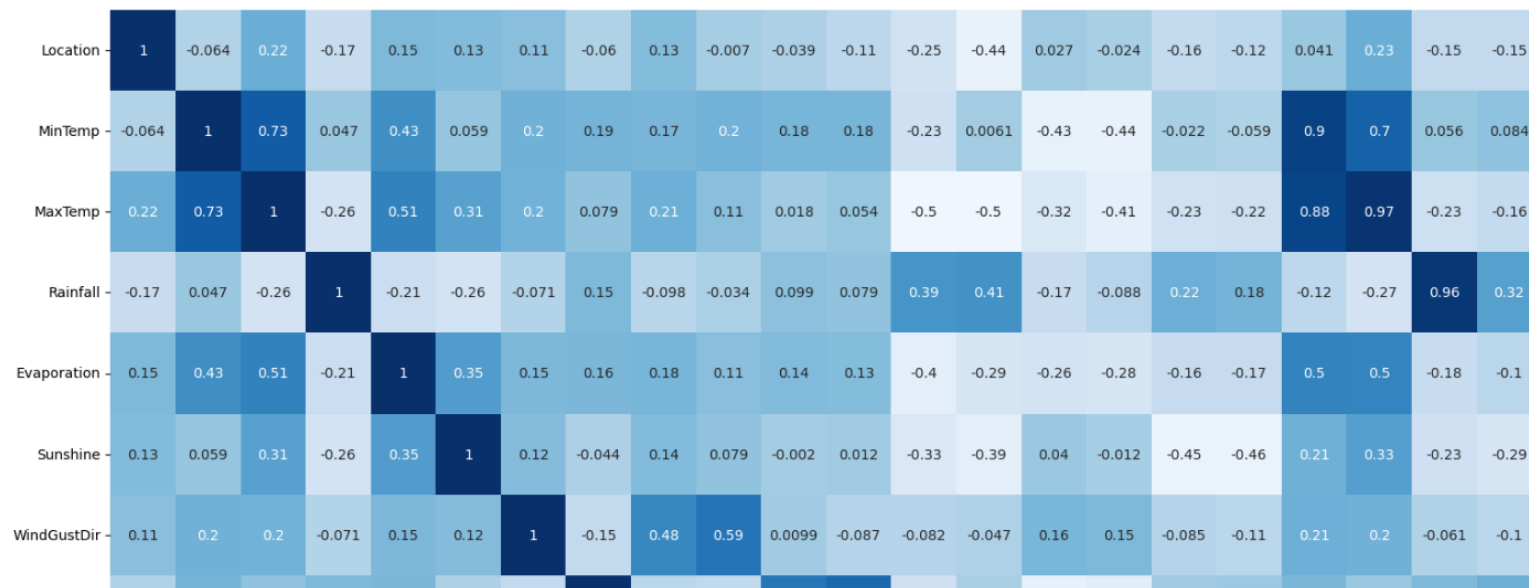
b. Barplot



c. Heatmap

```
[ ] plt.figure(figsize=(25,25))
    sns.heatmap(df.corr(),annot=True,cmap='Blues')
```

<Axes: >



6) To make model prediction and building better we assign numerical values to the location and the target variable

```
#have to encode location and date
df_new=df.groupby(['Location'])['RainTomorrow'].value_counts().sort_values(ascending=False).unstack()
df_new
```

```
location=df_new[1].sort_values(ascending=False).index
loc={}
for i in range(len(location)):
    loc[location[i]]=i
print(loc)
```

```
{'Portland': 0, 'Cairns': 1, 'Walpole': 2, 'Dartmoor': 3, 'MountGambier': 4, 'NorfolkIsland': 5, 'Albany': 6, 'Witchcliffe': 7, 'CoffsHarbour': 8, 'Sydney': 9}
```

```
[ ] df['Location']=df['Location'].map(loc)
```

```
[ ] df["Date"] = pd.to_datetime(df["Date"], format = "%Y-%m-%dT", errors = "coerce")
df["Date_month"] = df["Date"].dt.month
df["Date_day"] = df["Date"].dt.day
```

7) Outlier treatment

```
fig,axes=plt.subplots(5,3,figsize=(15,20))
fig.tight_layout(pad=3.0) #padding for better spacing
for i in range(len(continuous_val_attributes)):
    att=continuous_val_attributes[i]
    sns.boxplot(df[att],ax=axes[i//3,i%3])
    axes[i//3,i%3].set_title(att)
plt.show()
```

+ Code

+ Te

```
[ ] for i in continuous_val_attributes:
    q3=df[i].quantile(0.75)
    q1=df[i].quantile(0.25)
    iqr=q3-q1
    ll=q1-1.5*iqr
    ul=q3+1.5*iqr
    df.loc[df[i]>=ul,i]=ul
    df.loc[df[i]<=ll,i]=ll
```

```
[ ] fig,axes=plt.subplots(5,3,figsize=(15,20))
fig.tight_layout(pad=3.0) #padding for better spacing
for i in range(len(continuous_val_attributes)):
    att=continuous_val_attributes[i]
    sns.boxplot(df[att],ax=axes[i//3,i%3])
    axes[i//3,i%3].set_title(att)
plt.show()
```


8) Splitting the data into train and test: after observing the heatmap, feature selection is done and then the data is split accordingly

```
[ ] dfp=df
    del dfp['Date_day']
    del dfp['Date_month']

[ ] dfp['']

[ ] x=dfp[["Location","MaxTemp","Sunshine","WindGustSpeed","Humidity9am","Pressure3pm","Cloud9am","Cloud3pm"]]
    y=dfp['RainToday']
    print(x.shape)
    print(y.shape)

[ ] x.head(20)

[ ] from sklearn.model_selection import train_test_split

[ ] X_train,X_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=42)

[ ] from imblearn.over_sampling import RandomOverSampler

[ ] #since there is imbalance of data we will use overSampler
    ros = RandomOverSampler(sampling_strategy='minority',random_state=22)
    X_train,y_train=ros.fit_resample(X_train, y_train)
```

Milestone 3: Model Building

For any problem statement, it is a good practice to try predicting using multiple models as results will differ for each model. Each model also focuses on different parts of the dataset to predict.

Steps for model building:

- Import libraries needed for all the models you want to build

```
[61] from sklearn.svm import SVC
      from xgboost import XGBClassifier
      from sklearn.linear_model import LogisticRegression
      from sklearn import metrics

      import tensorflow as tf
      from tensorflow import keras
      from tensorflow.keras import layers
```

- Make an instance of each model

```

models = [LogisticRegression(), XGBClassifier()]
from sklearn.metrics import classification_report
for i in range(2):
    models[i].fit(X_train, y_train)
    print(models[i])
    train_preds = models[i].predict_proba(X_train)
    trainpred=models[i].predict(X_train)
    print('Training Accuracy : ', metrics.roc_auc_score(y_train, train_preds[:,1]))
    testpred=models[i].predict(X_test)
    val_preds = models[i].predict_proba(X_test)
    print('Validation Accuracy : ', metrics.roc_auc_score(y_test, val_preds[:,1]))
    report = classification_report(y_test,testpred)
    print(report)
    print()

~~~~~
model = keras.Sequential([
    layers.Input(shape=(X_train.shape[1],)),
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(64, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(32, activation='relu'),
    layers.Dense(1, activation='sigmoid')
])
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
early_stopping = keras.callbacks.EarlyStopping(monitor='val_loss', patience=10)
history = model.fit(X_train, y_train, epochs=50, batch_size=64, validation_split=0.2, callbacks=[early_stopping])
loss, accuracy = model.evaluate(X_test, y_test)
print(f"Test accuracy: {accuracy}")
predictions = model.predict(X_test)

```

- Compare accuracies and classification report to check which model performed the best
- Save the chosen model as a pickle file and download it

```

import pickle
pickle.dump(models[1],open('rainfall1.pk1','wb'))

```

Milestone 4: Application building

Steps:

- Create a project folder
- Open the folder in VSCode and create a virtual environment in it using the command “python -m venv nameofvirenv”
- Activate the virtual environment
- Pip install all the necessary libraries
- Create a templates folder, in that create a new file named “index.html”, this file be used for creation of the front-end UI. Add other routes to the interface while displaying if it will rain or not

```
templates > <> norain.html > ...
1  <!DOCTYPE html>
2  <html>
3  <head>
4  |   <title>No Rain</title>
5  </head>
6  <body>
7  |   <h1>No Rain, Enjoy Your Day!</h1>
8  |   
9  </body>
10 </html>
```

```
templates > <> rain.html > ...
1  <!DOCTYPE html>
2  <html>
3  <head>
4  |   <title>Rainy Day</title>
5  </head>
6  <body>
7  |   <h1>Chance of Rain, Carry an Umbrella</h1>
8  |   
9  </body>
10 </html>
```

```

1  <!DOCTYPE html>
2  <html>
3  <head>
4      <title>Rainfall Predictor</title>
5      <style>
6          body {
7              background-color: rgb(34, 113, 240);
8              font-family: Arial, sans-serif;
9              text-align: center;
10             margin: 0;
11             padding: 0;
12         }
13         h1 {
14             background-color: rgba(0, 0, 0, 0.7);
15             color: white;
16             padding: 10px;
17         }
18         form {
19             background-color: rgba(255, 255, 255, 0.8);
20             padding: 20px;
21             border-radius: 10px;
22             margin: 20px auto;
23             max-width: 400px;
24         }
25         label {
26             display: block;
27             margin-top: 10px;
28         }
29         input {
30             width: 100%;
31             padding: 10px;
32             margin-top: 5px;
33             margin-bottom: 15px;
34             border: 1px solid #ccc;
35             border-radius: 4px;
36         }
37
38         input[type="submit"] {
39             background-color: #007BFF;
40             color: white;
41             padding: 10px 15px;
42             border: none;
43             border-radius: 4px;
44             cursor: pointer;
45         }
46
47         input[type="submit"]:hover {
48             background-color: #0056b3;
49         }
50         p {
51             background-color: rgba(255, 255, 255, 0.8);
52             padding: 10px;
53             border-radius: 10px;
54             margin: 20px auto;
55             max-width: 400px;
56             font-weight: bold;
57         }
58     </style>
59 </head>
60 <body>
61     <h1>Rainfall Prediction</h1>
62     <form method="POST">
63         <label for="input1">Location</label>
64         <select name="s" id="cityDropdown"></select>
65         <label for="input2">Maximum Temperature(Celsius)</label>
66         <input type="number" name="input2" step="any" required><br>
67         <label for="input3">Sunshine</label>
68         <input type="number" name="input3" step="any" required><br>
69         <label for="input4">Wind Gust Speed</label>
70         <input type="number" name="input4" step="any" required><br>
71         <label for="input5">Humidity at 9am</label>
72         <input type="number" name="input5" step="any" required><br>
73         <label for="input6">Pressure at 3pm</label>
74         <input type="number" name="input6" step="any" required><br>
75         <label for="input7">Cloud at 9am</label>
76         <input type="number" name="input7" step="any" required><br>
77         <label for="input8">Cloud at 3pm</label>
78         <input type="number" name="input8" step="any" required><br>
79         <input type="submit" value="Predict">
80     </form>
81     <script>
82         const cityMapping= {'Portland':0,'Cairns': 1,
83             'Walpole': 2, 'Dartmoor': 3,
84             'MountGambier': 4, 'NorfolkIsland': 5,
85             'Albany': 6, 'Witchcliffe': 7,
86             'CoffsHarbour': 8, 'Sydney': 9, 'Darwin': 10,
87             'MountGinini': 11, 'NorahHead': 12, 'Ballarat': 13,
88             'GoldCoast': 14, 'SydneyAirport': 15, 'Hobart': 16, 'Watsonia': 17,
89             'Newcastle': 18, 'Wollongong': 19, 'Brisbane': 20,
90             'Williamstown': 21, 'Launceston': 22, 'Adelaide': 23,
91             'MelbourneAirport': 24, 'Perth': 25,
92             'Sale': 26, 'Melbourne': 27, 'Canberra': 28, 'Albury': 29,
93             'Penrith': 30, 'Nuriootpa': 31, 'BadgerysCreek': 32,
94             'Tuggeranong': 33, 'PerthAirport': 34, 'Bendigo': 35,
95             'Richmond': 36, 'WaggaWagga': 37, 'Townsville': 38,
96             'PearceRAAF': 39, 'SalmonGums': 40, 'Moree': 41,
97             'Cobar': 42, 'Mildura': 43, 'Katherine': 44,
98             'AliceSprings': 45, 'Nhil': 46, 'Woomera': 47, 'Uluru': 48}
99         const cityDropdown = document.getElementById('cityDropdown');
100         for (const city in cityMapping) {
101             const option = document.createElement('option');
102             option.value = cityMapping[city];
103             option.text = city;
104             cityDropdown.appendChild(option);
105         }
106     </script>
107
108     {% if prediction is not none %}
109     <p>
110         Prediction:
111         {% if prediction == 1 %}
112             It will rain. Carry an Umbrella!
113         {% else %}
114             No rain, Enjoy your Day!
115         {% endif %}
116     </p>
117     {% endif %}
118 </body>
119 </html>

```

index.html

- Create a back-end file called app.py which will help connect the UI to the model, receive the data and send back the prediction

```

app.py > predict
1  from flask import Flask, render_template, request, redirect, url_for
2  import pickle
3  import numpy as np
4  app = Flask(__name__)
5  with open("model.pkl", "rb") as model_file:
6      model = pickle.load(model_file)
7  @app.route("/", methods=["GET", "POST"])
8  def predict():
9      prediction = None
10     if request.method == "POST":
11         location = int(request.form["s"]) # Get the selected city as an integer
12         input_values = [float(request.form[f"input{i}"]) for i in range(2, 9)] # Updated field names
13         input_values.insert(0, location)
14         input_array = np.array(input_values).reshape(1, -1)
15         prediction = model.predict(input_array)[0]
16         if prediction==0:
17             return redirect(url_for('norain'))
18     else:
19         return redirect(url_for('rainy'))
20     return render_template("index.html", prediction=prediction)
21 @app.route("/norain")
22 def norain():
23     return render_template('norain.html')
24 @app.route("/rainy")
25 def rainy():
26     return render_template("rain.html")
27 if __name__ == "__main__":
28     app.run(debug=True)

```

- After all the files have been made, run the command “python app.py” on PowerShell and view the app.

Milestone 5: Final Application view

chances of rain today.



